ESP32 Firmware (esp/)

- **esp.ino**
  Main firmware entry point. Handles ROS communication loop, publishes encoder and pose data, subscribes to motor commands, and manages timing (publishing, debug, heartbeat). Includes direct motor control without onboard PID.
- **JOESWiFiSetup.h**
  WiFi and ROS connection configuration. Defines static IP setup, ROS master connection, and initialization of publishers/subscribers.
- **MotorBridge.h**
  Hardware abstraction header. Contains pin definitions, robot parameters (wheel radius, PPR, etc.), and function declarations for motor control and odometry.
- **MotorBridge.cpp**
  Hardware implementation layer. Handles:
  - Motor actuation (PWM + direction control)
  - Encoder interrupt processing
  - RPM calculation
  - Odometry estimation (x, y, θ)
- **WiFiHardware.h**
  Custom transport layer enabling ROS serial communication over WiFi (TCP). Acts as a bridge between ESP32 and ROS master.

📁 **ROS Package (ros/)**

scripts/

- **communication_tester.py**
  Diagnostic tool for validating bidirectional communication. Subscribes to all ESP32 topics and can send test messages and motor commands.
- **motor_commander.py**
  Interactive CLI tool for manual motor control using predefined commands or custom RPM inputs.
- **ros_pid_controller.py**
  Converts /cmd_vel commands into wheel RPMs. Intended for future closed-loop control (currently open-loop).

📦 ROS Configuration

- **CMakeLists.txt & package.xml**
  ROS1 (catkin) build and package configuration files.
  ⚠️ **Note:** These are configured for ROS1, not ROS2.

📘 Documentation

- **comms_explanation.ipynb**
  Comprehensive guide covering:
  - System architecture (ESP32 ↔ ROS over WiFi)
  - Topic structure and message formats
  - File responsibilities
  - Step-by-step testing procedure
  - Troubleshooting guide
    Serves as the main reference for setup and integration.

⚠️ Important Notes

- The current implementation is tested with **2 motors only**.
  ➤ System must be extended to support additional motors if required.
- Users must:
  - Recalculate **PPR (Pulses Per Revolution)** based on their encoders
  - Adjust **RPM mappings** according to motor characteristics

---

⚙️ Reference Motor Data (Graduation Project)

Provided calibration data for multiple motors including:

- PPR values (~1993–2003 pulses/rev)
- Pulse-to-length conversion (~0.097 mm/pulse)
- PWM ranges
- PID gains (Kp, Ki, Kd)

These values are **motor-specific** and should be recalibrated for different hardware setups.

🧪 **Testing**

- System validated through **manual testing**
- Verified:
  - ESP32 → ROS topic publishing
  - ROS → ESP32 command transmission
  - Motor actuation and encoder feedback
  - Debug and heartbeat signals